

UMAF: A Universal Multi-Agent Framework

Technical Report — v1.8

(Toy Version — Demonstrational Prototype)

Zhinan Li

zhinan.li@tum.de

Universal Multi-Agent Framework Project

June 2026

Abstract

Disclaimer: This is a toy version of a technical report generated by the UMAF demonstrational prototype. It serves as an illustration of the framework’s automated research pipeline capabilities and should not be considered a peer-reviewed publication.

This technical report presents UMAF (Universal Multi-Agent Framework), a LangChain + DeepSeek multi-agent system built on a 5-layer OOP class hierarchy with 32 specialized AgentRole subclasses and 7 domain-specific pipelines spanning code generation, research synthesis, skill detection, topology optimization, feature implementation, and autonomous self-evolution. The framework supports dual LLM backends—DeepSeek API via ChatOpenAI and Claude CLI via subprocess—with backend-aware task generation that adapts prompts to each backend’s capabilities. A centralized tool system provides 8 sandboxed capabilities through a ToolRegistry architecture, with tool assignments driven entirely by an externalized `tools_config.json` serving as the single source of truth. The system implements a layered defensive architecture: agent-level circuit breakers (force wrap-up, error spiral detection, post-loop forced write), pipeline-level stop-on-failure with dependency-aware topological execution, role-level honest success verification via `os.path.isfile()`, and framework-level deterministic fallback chains at every decision point. Evaluation across versions v1.4–v1.8 demonstrates 100% worker success rate, top review scores of 48/50, 60KB LaTeX output, and 379 passing behavioral tests. We provide a comprehensive comparison with AutoGen [1], CrewAI [4], and MetaGPT [2] across seven dimensions, and outline a Phase 3–5 roadmap for production readiness, advanced capabilities, and autonomous scaling.

Contents

1	Architecture Overview: 5-Layer OOP Class Hierarchy, 7 Pipelines, and 32 AgentRoles	5
1.1	Architectural Motivation	5
1.2	The 5-Layer OOP Class Hierarchy	5
1.3	The Seven Pipelines	5
1.3.1	CoderPipeline (v1.0)	5
1.3.2	ResearchPipeline (v1.2–v1.4)	5
1.3.3	CoderPPPPipeline (v1.4–v1.6.1)	6
1.3.4	TopologyPipeline (v1.5)	6

1.3.5	SkillPipeline (v1.5)	6
1.3.6	FeaturePipeline (v1.6)	6
1.3.7	SelfEvolutionPipeline (v1.8)	6
1.4	The 32 AgentRole Taxonomy	6
1.5	Resilience Patterns	7
2	LLM Backend System: DeepSeek API, Claude CLI Subprocess, and Backend-Aware Task Generation	8
2.1	Dual-Backend Architecture	8
2.2	DeepSeekProvider	8
2.3	ClaudeCLILLM	8
2.4	Backend-Aware Task Generation	8
2.5	The Pre-Fetch Layer	9
2.6	CheckpointManager and Conversation Logger	9
2.7	Backend Trade-offs	9
2.8	JSON Tool-Call Parsing Pipeline	9
3	Tool System Design: 8 Tools, ToolRegistry Architecture, and tools_config.json	10
3.1	Architectural Principles	10
3.2	The 8 Tool Implementations	10
3.3	ToolRegistry Architecture	10
3.4	tools_config.json: Single Source of Truth	10
3.5	Tool Name Translation for Claude CLI	11
3.6	The TOOL_MAP Separation Pattern	11
3.7	Tool Assignment Profiles	11
4	CoderPipeline and CoderPPPipeline: Single-File and Multi-File Code Generation	12
4.1	Overview	12
4.2	CoderPipeline: 2-Node Cyclic Graph	12
4.3	CoderPPPipeline: 5-Node Graph with Self-Loops	12
4.4	Decomposition Strategies	12
4.5	The v1.6.1 Workers Node Fix	13
4.6	Post-Reviewer Retry Classification	13
4.7	The REVIEW_PASSED/REVIEW_FAILED Token Scanning Pattern	13
4.8	Comparison with Related Work	13
5	ResearchPipeline: Dependency-Ordered Workers, Version-Bump Retries, and LaTeX Generation	14
5.1	Overview	14
5.2	Dynamic Decomposition with Complexity-Scaled Sizing	14
5.3	Dependency-Ordered Execution via Topological Levels	14
5.4	Version-Bump Retry with Context Reuse	14
5.5	Honest Success Reporting	15
5.6	The 5-Dimension Reviewer Scoring System	15
5.7	LaTeX Report Generation	15
5.8	Always-Forward Routing	15

6	TopologyPipeline and SkillPipeline: Agent Topology Optimization and Artifact-Agnostic Skill Detection	16
6.1	Overview	16
6.2	TopologyPipeline: 4-Node Linear Graph	16
6.2.1	TopologyAnalyzerRole: 6-Factor Assessment	16
6.2.2	TopologyDesignerRole: 4-Pattern Catalog	16
6.2.3	TopologyEvaluatorRole: 5-Dimension Scoring	16
6.2.4	TopologyWriterRole: Dual-Format Output	17
6.3	SkillPipeline: Fan-Out/Fan-In Graph	17
6.3.1	SkillScannerRole: 8-Category Classification	17
6.3.2	Four Artifact-Agnostic Detectors	17
6.3.3	SkillAggregatorRole and Output	17
6.4	Meta-Circular Validation	18
7	FeaturePipeline and SelfEvolutionPipeline: Project-Aware Code Generation and Autonomous Self-Improvement	19
7.1	Overview	19
7.2	FeaturePipeline: Scanner → Planner → Coder ↔ Reviewer → Writer	19
7.2.1	Graph and Routing	19
7.2.2	FeatureScannerRole: Project Context Generation	19
7.2.3	FeaturePlannerRole: Dual Create/Modify Planning	19
7.2.4	FeatureCoderRole and FeatureReviewerRole	19
7.3	SelfEvolutionPipeline: Analyzer → Planner → Coder ↔ Reviewer → Writer	19
7.3.1	Safety Architecture	19
7.3.2	SelfEvolutionAnalyzerRole	20
7.3.3	SelfEvolutionCoderRole: In-Place Modification	20
7.3.4	SelfEvolutionReviewerRole	20
7.4	Comparative Analysis Across All Code Generation Pipelines	20
8	Design Decisions and Engineering Practices: Circuit Breakers, Fallbacks, and Defensive Programming	21
8.1	Defensive AI Architecture Philosophy	21
8.2	The Autonomous Agent Loop with Three Circuit Breakers	21
8.2.1	Circuit Breaker 1: Force Wrap-Up	21
8.2.2	Circuit Breaker 2: Error Spiral Detection	21
8.2.3	Circuit Breaker 3: Unknown Tool Warnings	21
8.2.4	Post-Loop Forced Write (Salvage Mechanism)	21
8.3	Stop-on-Failure and Dependency Management	21
8.4	Catalog of Fallback Mechanisms	22
8.5	Double-Parse Anti-Pattern Prevention	22
8.6	Dependency Injection Fixes (v1.6.1)	22
8.7	Router Always Moves Forward	22
8.8	Python ≥ 3.11 and X None Syntax	22

9	Evaluation Results v1.4–v1.8: Quantitative Metrics and Version-by-Version Progress	23
9.1	Evaluation Philosophy: From Output Validation to Behavioral Verification	23
9.2	Test Infrastructure Architecture	23
9.3	Behavioral Test Design	23
9.4	Version-by-Version Quantitative Trajectory	24
9.5	Production Metrics	24
9.6	Dual-Layer Evaluation Architecture	24
9.7	v1.4.1 Agent Loop Reliability Foundations	24
10	Comparison with AutoGen, CrewAI, and MetaGPT, and Phase 3–5 Roadmap	25
10.1	Four Divergent Coordination Paradigms	25
10.2	Seven-Dimensional Comparison	25
10.2.1	Agent Communication Patterns	25
10.2.2	Tool Systems	25
10.2.3	LLM Backend Support	25
10.2.4	Pipeline/Graph Orchestration	26
10.3	The Conversation-to-Graph Spectrum	26
10.4	UMAF’s Unique Contributions	26
10.5	UMAF’s Limitations	26
10.6	Phase 3–5 Future Roadmap	27
10.6.1	Phase 3: Production Readiness (v2.0–v2.2)	27
10.6.2	Phase 4: Advanced Capabilities (v2.3–v2.5)	27
10.6.3	Phase 5: Scaling and Autonomy (v3.0+)	27
11	Scoring Summary	28

1 Architecture Overview: 5-Layer OOP Class Hierarchy, 7 Pipelines, and 32 AgentRoles

1.1 Architectural Motivation

UMAF (Universal Multi-Agent Framework) is a LangChain [5] + DeepSeek [51] multi-agent system built on a strict 5-layer OOP class hierarchy. The architecture is organized around three architectural backbones—**BaseAgent** (autonomous agent loop with dual backends), **AgentRole** (abstract template method defining agent lifecycle), and **BasePipeline** (LangGraph-based pipeline orchestrator)—that together form a composable, backend-agnostic framework for coordinating LLM-powered agents through complex, multi-stage workflows.

The architectural motivation is to separate concerns vertically (data → infrastructure → agent logic → specialized behavior → orchestration) and horizontally (seven independent pipelines for distinct use cases). This layering enables independent evolution of each concern.

1.2 The 5-Layer OOP Class Hierarchy

Layer 1 — Data Types: The foundation consists of **ToolSpec** (frozen dataclass with `name`, `description`, `parameters` for 8 canonical tools), **AgentResult** (holding `messages`, `iterations`, `success`), TypedDict State Types for each pipeline (7 distinct state schemas), and **CheckpointManager** for versioned checkpoint persistence.

Layer 2 — Infrastructure: The **LLMProvider ABC** provides a single `invoke(messages)` → **AIMessage** interface with two implementations: **DeepSeekProvider** (ChatOpenAI wrapper, model `deepseek-chat`, temperature 0.3, max_tokens 8192) and **ClaudeCLILLM** (subprocess-based `claude -p` with stream-json parsing, 600s timeout). The **ToolRegistry** centralizes 8 **ToolSpec** instances and 28 classmethods returning per-role tool lists.

Layer 3 — Agent Core: **BaseAgent** (~830 lines) implements the full autonomous agent loop [6, 9] with dual-backend dispatch, three circuit breakers (force wrap-up at max_steps-3, error spiral detection at 2+ consecutive persistent errors, unknown tool warnings), write reminders, post-loop forced write, and a 4-strategy JSON tool-call parser with state-machine-based repair. **AgentRole** (ABC) uses the Template Method pattern [45] with `tools_for_backend()`, `build_task()`, and `parse_result()` as overridable primitive operations, with `execute()` as the template method orchestrating the full lifecycle.

Layer 4 — Concrete Roles: 32 **AgentRole** subclasses spanning 7 pipelines, each implementing the three abstract methods with pipeline-specific behavior.

Layer 5 — Pipeline Classes: Seven **BasePipeline** subclasses, each implementing `_build_graph()`, `_build_initial_state()`, `_decompose()`, and `_print_results()` to compose roles into LangGraph workflows.

1.3 The Seven Pipelines

1.3.1 CoderPipeline (v1.0)

A minimal 2-node cyclic graph with max 5 iterations: **coder** ↔ **reviewer**. Coder generates code (6 tools), reviewer validates using `REVIEW_PASSED/REVIEW_FAILED` token scanning (5 tools, no `write_file`). The reviewer receives `coder_files` via `os.walk()` scan (v1.6.1 fix).

1.3.2 ResearchPipeline (v1.2–v1.4)

A 4-node graph with worker self-loop: **head** → **workers** ∅ → **reviewer** → **writer**. The head agent decomposes research topics into 2–8 sub-topics with complexity-scaled sizing. Workers

execute with dependency ordering via `_topological_levels()` and version-bump retry (max 6 versions, 5 retries) with checkpoint-based context reuse. The reviewer scores on 5 dimensions (depth, accuracy, relevance, clarity, originality, each 1–10). The writer generates LaTeX with `_latex_escape()` handling all 10 special characters and a template-based fallback.

1.3.3 CoderPPPPipeline (v1.4–v1.6.1)

A 5-node graph with self-loops: `head` $\rightarrow$ `workers` $\hookrightarrow$ `observer` $\rightarrow$ `reviewer` $\leftrightarrow$ `workers` $\rightarrow$ `organizer`. The head agent reads `.tex` and `.md` spec files and decomposes code generation into modules with dependency declarations. Workers implement modules in topological order with `_dependency_outputs` injection. The reviewer distinguishes worker failure (no code) from reviewer failure (code exists but has bugs). Post-reviewer retry classification prevents unnecessary regeneration. The organizer assembles only passed modules.

1.3.4 TopologyPipeline (v1.5)

A 4-node strict linear graph with no loops: `analyzer` $\rightarrow$ `designer` $\rightarrow$ `evaluator` $\rightarrow$ `writer`. The analyzer assesses task complexity across 6 factors. The designer proposes 2–4 candidate topologies using 4 patterns (sequential, `fan_out_fan_in`, `debate_consensus`, hierarchical). The evaluator scores on 5 dimensions (latency, reliability, `cost_efficiency`, simplicity, scalability, each 1–10). The writer produces `topology_spec.json` and `topology_report.md`.

1.3.5 SkillPipeline (v1.5)

A 4-node fan-out/fan-in graph: `scanner` $\rightarrow$ [4 parallel detectors] $\rightarrow$ `aggregator` $\rightarrow$ `writer`. The scanner classifies artifacts into 8 types using deterministic scoring. Four artifact-agnostic detectors (DomainExpertise, TechnicalCraft, Methodology, Rigor) run in parallel, each reading the same `artifact_analysis.json` but extracting different skill dimensions. The aggregator deduplicates and categorizes. The writer produces `skills.json` and `skills_report.md`.

1.3.6 FeaturePipeline (v1.6)

A 5-node graph with coder $\leftrightarrow$ reviewer cycle (max 5 iterations): `scanner` $\rightarrow$ `planner` $\rightarrow$ `coder` $\leftrightarrow$ `reviewer` $\rightarrow$ `writer`. The scanner surveys the project directory to produce `project_context.json` with language, conventions, and file manifest. The planner generates `implementation_plan.json` with both `files_to_create` and `files_to_modify`. The coder reads existing files before modification and matches project conventions. The reviewer validates across 4 dimensions (completeness, correctness, convention compliance, test quality).

1.3.7 SelfEvolutionPipeline (v1.8)

A 5-node graph with coder $\leftrightarrow$ reviewer cycle (max 3 iterations): `analyzer` $\rightarrow$ `planner` $\rightarrow$ `coder` $\leftrightarrow$ `reviewer` $\rightarrow$ `writer`. The analyzer scans UMAF’s own codebase and agent logs to identify improvement opportunities across 6 categories. The planner creates concrete implementation plans with `current_code_snippet` and `new_code`. The coder modifies source in-place with git-diff-based change detection. The reviewer runs `pytest test/` for regression verification. Safety: operates on current git branch with `git checkout - .` reversibility.

1.4 The 32 AgentRole Taxonomy

All 32 roles inherit from `AgentRole` (ABC) and override `tools_for_backend()`, `build_task()`. Some override `parse_result()` for structured output extraction. The roles span 7 pipelines:

CoderPipeline (2 roles), ResearchPipeline (4), CoderPPPPipeline (5), TopologyPipeline (4), SkillPipeline (7), FeaturePipeline (5), SelfEvolutionPipeline (5).

1.5 Resilience Patterns

UMAF employs seven layers of resilience [46]: (1) circuit breakers in BaseAgent preventing infinite loops, (2) checkpoint-based retry for failed workers, (3) stop-on-failure in dependency graphs preventing cascading failures, (4) fallback at every level (decomposition, scoring, LaTeX, skill detection), (5) dual-backend transparency through factory pattern, (6) always-forward routing accepting `researched_partial` results, and (7) merge-on-completion providing consolidated audit trails.

2 LLM Backend System: DeepSeek API, Claude CLI Subprocess, and Backend-Aware Task Generation

2.1 Dual-Backend Architecture

UMAF’s LLM layer implements a dual-backend architecture providing transparent access to two fundamentally different invocation paradigms: a traditional API-based approach via DeepSeek’s OpenAI-compatible endpoint, and a subprocess-based approach that shells out to the Claude Code CLI. The architecture unifies both backends behind a common `LLMProvider ABC` with a single `invoke(messages) → AIMessage` interface.

2.2 DeepSeekProvider

The DeepSeek backend [51] wraps LangChain’s `ChatOpenAI` pointed at `https://api.deepseek.com/v1` with model `deepseek-chat`, temperature 0.3, and `max_tokens=8192`. API key resolution uses `DEEPSEEK_API_KEY` from `.env` via `python-dotenv`. The `invoke()` method delegates directly to LangChain’s `ChatOpenAI.invoke()`, which handles OpenAI-compatible request/response serialization, retry, and error mapping.

2.3 ClaudeCLILLM

The Claude CLI backend [26] constructs a `claude -p <prompt>` subprocess with `-output-format stream-json`, `-verbose`, and `-permission-mode bypassPermissions`. Key features include:

- **Timeout:** Default 600s, enforced via `threading.Timer` that kills the subprocess on timeout
- **Environment injection:** Merges 12 env vars from `claude_env_sample.json`
- **Working directory:** Accepts `cwd` parameter for path resolution
- **Allowed tools:** Per-role tool restriction via `-allowedTools`
- **Two invocation modes:** `invoke()` for text output, `stream_invoke()` for stream-json event processing

2.4 Backend-Aware Task Generation

This is UMAF’s most important architectural innovation. Every `AgentRole.build_task(backend, ...)` receives the backend identifier and tailors the prompt accordingly at three layers:

Layer 1 — BaseAgent Prompt Builders: DeepSeek receives a 311-line prompt with JSON tool-call formatting rules, 4 escape rules, and a write strategy hierarchy (prefer `run_command` Python one-liners over `write_file`). Claude CLI receives a 30-line prompt with native tool instructions.

Layer 2 — AgentRole Per-Backend Branching: For DeepSeek: emphasizes `call_claude` for deep reasoning, uses `download_file` + `read_file` for arxiv.org. For Claude CLI: the agent IS a Claude Code instance and must NOT spawn nested `claude -p` calls.

Layer 3 — BaseDecomposerRole Instructions: Head agents append backend-specific suffixes. Claude CLI agents are told “Use your own knowledge—do NOT search the web.” DeepSeek agents receive web search instructions and arxiv.org pre-fetch patterns.

2.5 The Pre-Fetch Layer

Claude Code’s cc-switch blocks domain verification for arxiv.org. UMAF’s pre-fetch layer downloads academic content at the framework level via Python `urllib` before agents run. The mechanism: (1) search for subtask via `web_search()`, (2) extract arxiv.org URLs via regex, (3) download up to 3 via `download_file()` to local HTML, (4) inject file paths into Claude CLI worker prompts with “Read them directly” instructions.

2.6 CheckpointManager and Conversation Logger

`CheckpointManager` provides versioned state persistence at `agent_log/{name}_v{version:02d}_checkpoint.json`. The `load_previous(n)` method enables cross-version context reuse for retry—loading the highest version below n with full message history restoration. The conversation logger writes timestamped JSON logs for debugging, performance analysis, and self-evolution feedback.

2.7 Backend Trade-offs

Table 1: DeepSeek vs. Claude CLI Comparison

Dimension	DeepSeek (ChatOpenAI)	Claude CLI (Subprocess)
Tool Calling	JSON parsing from text (4-strategy parser)	Native tool calling via stream-json
Reliability	Moderate—prone to JSON malformation	High—structurally validated by CLI
Multi-Step Tasks	Gets stuck in loops, loses state	Maintains coherent state across contexts
Cost	~\$0.28/M input, ~\$0.42/M output	~\$3/M input, ~\$15/M output
Latency	Fast—single API call per iteration	Slower—subprocess startup overhead
Circuit Breakers	Full intervention system	Limited to hard timeout + auto-retry
Checkpointing	After every tool execution	After every stream-json event

2.8 JSON Tool-Call Parsing Pipeline

Since DeepSeek lacks native tool calling [51,52], UMAF implements a 4-strategy parser:

1. Markdown code fences (reversed order—newest first)
2. Standard JSON key order (`{"tool": ..., "args": ...}`)
3. Reversed key order (`{"args": ..., "tool": ...}`)
4. JSON repair: remove trailing commas, escape raw whitespace via state machine

A write strategy hierarchy mitigates JSON escaping: prefer `run_command` with Python one-liner, then `write_lines`, then `write_file` as last resort.

3 Tool System Design: 8 Tools, ToolRegistry Architecture, and tools_config.json

3.1 Architectural Principles

UMAF’s tool system follows three architectural principles [26, 55]: **separation of specification from implementation** (ToolSpec dataclasses describe what a tool does; TOOL_MAP callables implement how), **centralized role-based access control** (28 classmethods in ToolRegistry return per-role tool lists driven entirely by a single JSON file), and **backend-transparent tool translation** (the same tools_for_backend() call produces DeepSeek JSON schemas or Claude CLI native tool names).

3.2 The 8 Tool Implementations

Table 2: Complete Tool Specifications

Tool	Parameters	Timeout	Description
read_file	path: str	N/A	Read file relative to working_dir
write_file	path, content: str	N/A	Write single-string content
write_lines	path: str, lines: list[str]	N/A	Join lines with newlines (avoids JSON escaping)
run_command	command: str, description: str (opt.)	30s	Shell execution via subprocess.run
call_claude	prompt: str	120s (config. 600s)	Spawn claude -p subprocess
web_search	query: str, max_results: int (opt.)	15s	DuckDuckGo Lite scraping
web_fetch	url: str, max_chars: int (opt.)	20s	urllib fetch with HTML stripping
download_file	url, output_path: str	30s	urllib download to local file

3.3 ToolRegistry Architecture

The ToolRegistry class provides 28 role-specific classmethods (all defaulting to []) plus utility methods:

- `set_tool_config(config)`: Class-level override injection from JSON
- `_apply_override(pipeline, role, defaults)`: Case-insensitive role matching
- `to_dicts(specs)`: Convert ToolSpec list to dict for prompt injection
- `get_map()`: Return copy of TOOL_MAP for BaseAgent initialization

The zero-default guarantee means that without explicit JSON configuration, agents receive zero tools—a safety-by-design decision ensuring no accidental tool leakage.

3.4 tools_config.json: Single Source of Truth

The JSON config file (236 lines) serves as the declarative, auditable definition of per-role tool assignments:

- **Structure**: Pipeline names as top-level keys, each mapping role names to tool name arrays

- **`__global__` fallback:** Universal fallback for unlisted roles
- **`__timeouts__` overrides:** Per-tool timeout configuration (e.g., `call_claude: 600`)
- **Metadata stripping:** Keys prefixed with `_` (except `__global__`) are documentation-only
- **Case-insensitive matching:** Substring matching on method suffixes
- **CLI override:** `-tools-config` flag for alternative config files

3.5 Tool Name Translation for Claude CLI

When the backend is `claude_cli`, Python tool names are translated to Claude CLI native names via two mechanisms: (1) `_CLAUDE_NATIVE_TOOL_SPECS` dict mapping each Python name to a `(native_name, description)` tuple, and (2) regex-based text replacement via `_translate_task_for_claude()`. The `-allowedTools` flag enforces restrictions at the CLI level.

3.6 The `TOOL_MAP` Separation Pattern

`TOOL_MAP` maps tool name strings to implementation callables, providing three benefits: spec/impl decoupling (`ToolSpec` in `registry.py`, implementations in `functions.py`), testability (mock replacements via `BaseAgent` init), and lazy binding (assignment at import time prevents circular imports).

3.7 Tool Assignment Profiles

Tool profiles vary significantly by role and pipeline. For example, `CoderPipeline`’s coder gets 6 tools (including `web_search` and `web_fetch` for research during coding), while its reviewer gets 5 tools (no `write_file`). `SelfEvolution`’s reviewer gets 3 tools (can write files for in-place fixes), while `FeaturePipeline`’s reviewer gets only 2 (`read_file` and `run_command`—critique-only). The most restricted role is the topology writer with only `write_file`.

4 CoderPipeline and CoderPPPipeline: Single-File and Multi-File Code Generation

4.1 Overview

UMAF provides two distinct code generation pipelines representing complementary approaches. **CoderPipeline** is a minimal 2-node cyclic graph for single-file tasks, where a coder generates and a reviewer validates through iterative review (max 5 cycles). **CoderPPPipeline** is a 5-node graph implementing a decompose→implement→review→assemble workflow for multi-file projects with explicit dependency management.

4.2 CoderPipeline: 2-Node Cyclic Graph

The StateGraph uses a **MultiAgentState** TypedDict with 8 fields. The routing logic: if review passes → END; if iteration ≥ 5 → END; otherwise route to the other agent (coder↔reviewer cycle). The entry point is always coder.

CoderFiles Injection (v1.6.1 Fix): Previously, the reviewer evaluated code without knowing which files the coder produced. The fix performs an `os.walk()` scan after coder completion, collects non-hidden files, and passes them as `coder_files` to the reviewer’s `execute()`, rendered in a “Files Produced by Coder” section (truncated at 50 entries).

4.3 CoderPPPipeline: 5-Node Graph with Self-Loops

The StateGraph has 5 nodes (head, workers, observer, reviewer, organizer) with status-based routing:

- `decomposed` → `workers`
- `worker_all_success` → `observer`
- `worker_retry` → `workers` (self-loop)
- `observed` → `reviewer`
- `reviewed_all_passed` → `organizer`
- `reviewed_retry` → `workers` (loop back)

4.4 Decomposition Strategies

CoderPipeline uses no decomposition—the requirement is passed directly as a single task. CoderPPPipeline’s head agent (**CoderPPDecomposerRole**, inheriting from **BaseDecomposerRole**) reads `.tex` or `.md` spec files and decomposes into sub-modules with explicit dependency declarations, environment setup (recording Python path, conda environment, pip packages), and a JSON schema with `id`, `module_name`, `description`, `files_to_create`, and `dependencies`.

The fallback decomposition: for LaTeX, extracts `\section{...}` titles as module ideas; for non-LaTeX, splits on commas and keywords. Always appends a `main` entry point and guarantees ≥ 2 modules.

4.5 The v1.6.1 Workers Node Fix

Prior to v1.6.1, `_workers_node` bypassed `_run_workers_with_deps()` entirely, running workers in flat parallel with no dependency awareness. The fix implements dependency resolution directly:

- A `completed` dict maps both `sub_task_id` (int) and `module_name` (str) to worker outputs
- Before each topological level, tasks with dependencies have `_dependency_outputs` injected
- Topological ordering via `_topological_levels()` with cycle detection and breaking
- File content validation: files under 100 bytes flagged as “empty/skeletal”
- Stop-on-failure: downstream tasks deferred on upstream failure

4.6 Post-Reviewer Retry Classification

A unique resilience feature distinguishes between worker failure (no `.py` files beyond `__init__.py` and test files—needs code regeneration) and reviewer failure (code exists but has bugs—just needs re-checking). This prevents unnecessary regeneration of code that merely needs re-review.

4.7 The REVIEW_PASSED/REVIEW_FAILED Token Scanning Pattern

Both pipelines use `scan_review_verdict()` from `utils.py`, which scans messages in reverse order (newest first), filters to `AIMessage` objects only, and uses exclusive PASS detection (`REVIEW_PASSED` only recognized when `REVIEW_FAILED` is NOT in the same message). CoderPPPipeline extends this with a double-check mechanism that also reads `review.md` as an authoritative source.

4.8 Comparison with Related Work

CoderPipeline’s coder↔reviewer loop implements the **Self-Refine** pattern [7]. CoderPPPipeline’s topology maps to **AgentMesh**’s planner→coder→debugger→reviewer pipeline [18], **DocAgent**’s topological processing order [19], and **ProjectGen**’s architecture design → skeleton generation → code filling stages [21]. Related approaches include CodeCoR’s self-reflective collaboration [17], See-Saw’s recursive multi-file generation [20], and Blueprint2Code’s planning-repair pipeline [43].

5 ResearchPipeline: Dependency-Ordered Workers, Version-Bump Retries, and LaTeX Generation

5.1 Overview

The ResearchPipeline is UMAF’s most architecturally sophisticated pipeline, implementing a 4-node LangGraph workflow [22,25] (`head` $\rightarrow$ `workers` $\rightarrow$ `reviewer` $\rightarrow$ `writer`) that transforms unstructured research topics into scored, ranked, LaTeX-formatted proposals. Three interlocking resilience mechanisms address the fundamental reliability problem in autonomous LLM research agents: dependency-ordered parallel execution with stop-on-failure, version-bump retry with checkpoint-based context reuse, and honest success reporting via `os.path.isfile()`.

5.2 Dynamic Decomposition with Complexity-Scaled Sizing

The head agent uses a three-tier fallback chain:

Tier 1 — LLM Decomposition: The decomposer inherits from `BaseDecomposerRole` and overrides five template methods. The `_sizing_guide()` provides complexity heuristics: narrow topics $\rightarrow$ 3–5 sub-topics, moderate $\rightarrow$ 5–8, broad $\rightarrow$ 8–12.

Tier 2 — Timeout Protection: The head node wraps the decomposition in a `ThreadPoolExecutor` with `HEAD_TIMEOUT=300s`.

Tier 3 — Programmatic Fallback: Splits on commas, “and”, “vs”, semicolons; creates a deep-dive subtopic per keyword; appends synthesis sub-topics; guarantees ≥ 2 sub-topics.

5.3 Dependency-Ordered Execution via Topological Levels

`BasePipeline._topological_levels()` transforms a flat task list with dependencies into execution levels using a modified Kahn’s algorithm—a pattern inspired by the MapReduce execution model [10] and extended by agentic MapReduce approaches [23,24]: key resolution (dependencies can be int IDs or string names), level construction (all independent tasks grouped), cycle detection and breaking (iterative edge removal prioritizing last dependencies), and fallback to parallel execution for unbreakable cycles.

`_run_workers_with_deps()` executes levels sequentially, injects `_dependency_outputs` into dependent tasks, and implements stop-on-failure: if any task fails and downstream levels remain, execution breaks immediately, deferring downstream dependents for retry.

5.4 Version-Bump Retry with Context Reuse

The workers node implements a three-mode state machine:

Mode A (Fresh Start): Run all tasks at version 1.

Mode B (Retry Only Failed): Identify workers with no output file, bump version by 1, run only failed workers, preserve successful outputs.

Mode C (Max Retries Exceeded): Final attempt, remaining failures get placeholder entries, status set to `researched_partial`.

`CheckpointManager.load_previous(current_version)` restores full message history from the previous attempt, resets iteration counter to 0, and injects a context-aware retry message.

Constants: `RESEARCH_MAX_VERSIONS=6`, `RESEARCH_MAX_WORKER_RETRIES=5`, `WORKER_TIMEOUT=900s`.

5.5 Honest Success Reporting

`ResearchWorkerRole.parse_result()` checks `os.path.isfile()` before reporting success—a binary gate: the worker either has an output file (success) or doesn’t (failure). This was the single most impactful reliability fix (v1.4), improving worker success rate from 66.7% to 100%.

5.6 The 5-Dimension Reviewer Scoring System

The reviewer scores each work on: depth, accuracy, relevance, clarity, originality (each 1–10, max 50 total). A three-tier extraction ensures output: (1) read `scoring_report.json`, (2) parse agent messages, (3) programmatic ranking by file existence and summary length with uniform 5/10 scores.

5.7 LaTeX Report Generation

The writer uses a sectioned output strategy: each research work is a separate `.tex` file with `\input{}` commands in a main file, avoiding monolithic write truncation. This approach is informed by the agent laboratory paradigm [16] and automated scientific writing pipelines [25]. `_latex_escape()` handles all 10 LaTeX special characters. `_fallback_latex()` generates a complete document from a pre-defined template when LLM generation fails.

5.8 Always-Forward Routing

The flow dict encodes a “progress over perfection” philosophy: `researched_partial` → `reviewer` routes partial worker results to the reviewer rather than aborting. Only three truly unrecoverable states abort the pipeline: no sub-tasks, no reviewable outputs, and no scored works.

6 TopologyPipeline and SkillPipeline: Agent Topology Optimization and Artifact-Agnostic Skill Detection

6.1 Overview

The TopologyPipeline and SkillPipeline represent UMAF’s meta-analysis layer. The TopologyPipeline implements a meta-cognitive capability: analyzing arbitrary multi-agent tasks, proposing candidate topologies using four canonical patterns, evaluating them across five dimensions, and producing executable specifications. The SkillPipeline implements a fan-out/fan-in graph where a scanner classifies artifacts and four parallel detectors analyze skills across independent dimensions.

6.2 TopologyPipeline: 4-Node Linear Graph

The pipeline follows a strict linear workflow with no cycles: `analyzer` $\rightarrow$ `designer` $\rightarrow$ `evaluator` $\rightarrow$ `writer`.

6.2.1 TopologyAnalyzerRole: 6-Factor Assessment

The analyzer assesses input tasks across six orthogonal dimensions: `data_dependencies` (inter-agent data flow), `parallelism_opportunities` (independent sub-tasks), `tool_requirements` (distinct tools per agent), `error_domains` (distinct failure modes), `latency_sensitivity` (real-time vs. batch), and `scale` (sub-task count and workload type). Each factor is assigned a level (low/medium/high) with reasoning. The fallback uses word count heuristics with conservative defaults.

6.2.2 TopologyDesignerRole: 4-Pattern Catalog

Four canonical multi-agent topologies [11–13]:

1. **Sequential**: Fixed-order linear chain. Best for strictly ordered workflows.
2. **Fan-Out/Fan-In**: Dispatcher $\rightarrow$ parallel workers $\rightarrow$ collector. Best for embarrassingly parallel sub-tasks.
3. **Debate/Consensus**: Multiple independent solvers $\rightarrow$ judge synthesizes. Best for high-stakes decisions.
4. **Hierarchical**: Orchestrator $\rightarrow$ domain leads $\rightarrow$ workers. Best for complex multi-layered tasks.

Each topology specifies agent configurations, connections, parallelism strategy, strengths, and weaknesses. The fallback generates three default topologies (sequential, fan-out/fan-in, hierarchical) with pre-configured agents.

6.2.3 TopologyEvaluatorRole: 5-Dimension Scoring

Scores each topology 1–10 on: latency (critical path length), reliability (fault tolerance), `cost_efficiency` (total tokens/calls), simplicity (implementation ease), and scalability (growth handling). The fallback heuristic encodes deliberate tradeoffs: fan-out/fan-in scores highest (36/50), `debate_consensus` lowest (26/50) despite highest reliability.

6.2.4 TopologyWriterRole: Dual-Format Output

Produces `topology_spec.json` (recommended topology with implementation guide and ASCII flow diagrams) and `topology_report.md` (comparison table, recommendation rationale, implementation notes). The writer has only `write_file`—the most restricted role in UMAF.

6.3 SkillPipeline: Fan-Out/Fan-In Graph

The graph topology directly mirrors the fan-out/fan-in pattern [3] that the TopologyPipeline’s evaluator scores highest: `scanner` → [4 `parallel detectors`] → `aggregator` → `writer`.

6.3.1 SkillScannerRole: 8-Category Classification

The scanner performs four phases: (1) surface scan via `find` and `ls`, (2) artifact classification into 8 types (`software_project`, `research_paper`, `blog_article`, `documentation`, `dataset`, `design_document`, `presentation`, `configuration`) using weighted scoring from multiple evidence sources, (3) deep reading of up to 12 representative files, (4) structure analysis. The `_classify_artifact()` algorithm computes weighted scores from source file ratio, extension matches, build file matches, and keyword matches—operating fully deterministically without LLM involvement.

6.3.2 Four Artifact-Agnostic Detectors

Each detector reads the same `artifact_analysis.json` but asks fundamentally different questions:

- **DomainExpertiseDetectorRole:** “What does the creator know deeply about?” Uses `_DOMAIN_SIGNALS` catalog across 9 pre-defined domains with indicator terms and proficiency heuristics (≥ 5 matches → “advanced”).
- **TechnicalCraftDetectorRole:** “How skilled is the creator at the medium itself?” Adaptive behavior: switches between code craft detection (design patterns, error handling, type systems) and writing craft detection (argumentation, technical writing) based on artifact type.
- **MethodologyDetectorRole:** “What tools, workflows, and processes are evident?” [28, 42] Covers 35+ tools across categories (languages, testing, CI/CD, containers, documentation, data science). File signal weighting: file matches count double in proficiency calculations.
- **RigorDetectorRole:** “How thorough, careful, and complete is the work?” Uses quantitative metrics: test file ratio, documentation ratio, quality config presence. Detects linting/formatting tools and academic rigor indicators (citations, methodology sections, reproducibility).

6.3.3 SkillAggregatorRole and Output

The aggregator merges four detector reports with name-based deduplication (keep highest proficiency), category normalization (tools mapped via `_CATEGORY_MAP`, skills via `_SKILL_CATEGORY_MAP`), and summary statistics. The writer produces `skills.json` (machine-readable) and `skills_report.md` (human-readable with proficiency badges and category emojis).

6.4 Meta-Circular Validation

Both pipelines were originally generated by UMAF's own CoderPP pipeline—a recursive self-application validating the framework's code generation capabilities. The TopologyPipeline optimizing topologies and the SkillPipeline detecting skills form a closed validation loop: the framework's own analysis tools confirm its architectural choices.

7 FeaturePipeline and SelfEvolutionPipeline: Project-Aware Code Generation and Autonomous Self-Improvement

7.1 Overview

FeaturePipeline (v1.6) and SelfEvolutionPipeline (v1.8) extend UMAF beyond greenfield code generation into brownfield development [41, 42] and meta-cognitive self-improvement [14, 33, 40]. FeaturePipeline grounds every agent action in a detailed `project_context.json`, enabling surgical edits to existing codebases. SelfEvolutionPipeline inverts the framework’s gaze: UMAF analyzes and improves itself through a structured `analyze→plan→implement→review` pipeline.

7.2 FeaturePipeline: Scanner → Planner → Coder ↔ Reviewer → Writer

7.2.1 Graph and Routing

UMAF’s most complex routing uses three separate routers: a linear status router for `scanner→planner` segments, a dedicated coder router, and a reviewer router implementing the core review loop (`review_passed → writer`; `iteration < 5 → coder`; `iteration ≥ 5 → writer`).

7.2.2 FeatureScannerRole: Project Context Generation

The scanner is the gateway to project-aware development with 5 steps: structure discovery via `find`, configuration analysis (parsing build files), convention extraction (reading 3–5 source files for naming, imports, types, docstrings, error handling), test pattern detection, and `project_context.json` generation. The 117-line deterministic fallback performs complete project analysis without LLM calls—completing in ~30s for a 2000-file project.

7.2.3 FeaturePlannerRole: Dual Create/Modify Planning

The key innovation: planning for both `files_to_create` and `files_to_modify`. Modification entries specify `path`, `section` (surgical targeting within files), `change`, and `description`. Validation enforces no circular dependencies and mandatory existing-file prereading.

7.2.4 FeatureCoderRole and FeatureReviewerRole

The coder follows a 5-step ordered procedure: read plan and context, modify existing files FIRST, create new files, write tests, run tests and fix. The reviewer evaluates across 4 dimensions (completeness, correctness, convention compliance, test quality) with only 2 tools (`read_file`, `run_command`)—the most restricted reviewer profile, enforcing critique-only review.

7.3 SelfEvolutionPipeline: Analyzer → Planner → Coder ↔ Reviewer → Writer

7.3.1 Safety Architecture

Seven layers of safety for self-modification [15, 44]: (1) git reversibility via `git checkout - .`, (2) reduced iteration limit (`MAX_ITERATIONS=3` vs. 5 for other pipelines), (3) test suite as gate (`pytest test/`), (4) change traceability via `git diff`, (5) fallback at every level, (6) no new dependencies, and (7) backward compatibility enforcement.

7.3.2 SelfEvolutionAnalyzerRole

Unique among all 32 roles: the only role that analyzes UMAF itself. Identifies improvement opportunities across 6 categories: prompt quality, parameter tuning, error handling, code quality, test gaps, and configuration. The fallback is notable for including a pre-seeded improvement suggestion (SEO-001: expand test coverage) that produced the v1.8 175-test expansion.

7.3.3 SelfEvolutionCoderRole: In-Place Modification

The coder operates on UMAF’s actual source tree with a three-tier change detection strategy: (1) `git diff -name-only` for modified tracked files, (2) `git ls-files -others -exclude-standard` for new files, (3) mtime-based fallback with 60-second window for recently-modified `.py` files. This approach is informed by self-evolving agent systems [37–39]. All changes follow UMAF conventions: Python ≥ 3.11 syntax, `from __future__ import annotations`, no multi-line docstrings.

7.3.4 SelfEvolutionReviewerRole

The gatekeeper mandates: run `pytest test/ -q`, check all tests pass, review code quality, output `REVIEW_PASSED` or `REVIEW_FAILED`. Extends `scan_review_verdict()` with regex-based test result extraction and early failure detection via `result.success` check.

7.4 Comparative Analysis Across All Code Generation Pipelines

Table 3: Four-Pipeline Code Generation Spectrum

Dimension	Coder	CoderPP	Feature	SelfEvolution
Dev. type	Greenfield (single)	Greenfield (multi)	Brownfield	Self-modification
Entry point	Coder	Head	Scanner	Analyzer
Project awareness	None	Spec files	<code>project_context.json</code>	Own codebase
Max cycles	5	5 versions	5	3
Change detection	<code>os.walk()</code>	File existence	Plan-based check	<code>git diff</code> + mtime
Safety	Working dir	Assembly dir	Read-only context	<code>git checkout - .</code>

Together, these four pipelines form a comprehensive code generation capability spanning single-file scripts to self-improving systems.

8 Design Decisions and Engineering Practices: Circuit Breakers, Fallbacks, and Defensive Programming

8.1 Defensive AI Architecture Philosophy

UMAF’s engineering philosophy treats LLMs as non-deterministic, potentially unreliable microservices [49], wrapping them with the same protective infrastructure used for distributed systems (circuit breakers, bulkheads, retries, fallback chains) [46], augmented with LLM-specific guards [30, 31]. The framework’s three-layer resilience architecture—agent-level circuit breakers, pipeline-level retry mechanisms, and framework-level fallback chains—implements the principle that **every stage must have a programmatic fallback**.

8.2 The Autonomous Agent Loop with Three Circuit Breakers

8.2.1 Circuit Breaker 1: Force Wrap-Up

Activates when `steps_left ≤ _FORCE_WRAPUP_THRESHOLD` (last 3 steps). Bifurcated prompt: if files exist—verify and declare `TASK_COMPLETE`; if no files—write best-effort output immediately, do not research further. The 3-step runway accommodates the LLM’s 2-step response pattern (think → act) with one safety margin.

8.2.2 Circuit Breaker 2: Error Spiral Detection

Terminates when `consecutive_errors ≥ 2` (tightened from 3 in v1.4.1). Error classification uses `_PERSISTENT_ERRORS` patterns (timeout, not found, permission denied, connection refused, invalid, error). Empirically validated: of 31 agents reaching 3 consecutive errors in v1.4, only 6.5% self-corrected. At threshold 2, 93.5% of doomed agents are caught while saving ~1 iteration of LLM calls per spiral.

8.2.3 Circuit Breaker 3: Unknown Tool Warnings

Tracks hallucinated tool names via `known_unavailable` set. Batch-reports at iteration start rather than individually, preventing prompt bloat. Addresses the common LLM failure mode of hallucinating tool names (e.g., `search_web` instead of `web_search`).

8.2.4 Post-Loop Forced Write (Salvage Mechanism)

After the agent loop exhausts all steps without `TASK_COMPLETE`, makes up to 2 additional LLM calls to salvage the task. Approximately 15–20% of otherwise-failed runs are salvaged by the first call, and an additional 5% by the second.

8.3 Stop-on-Failure and Dependency Management

`_run_workers_with_deps()` implements the stop-on-failure pattern adapted for LLM agent DAGs: after each topological level, if any task fails and downstream levels remain, execution breaks immediately. Dual-key registration in the `completed` dict (by `sub_task_id` and `module_name`) supports heterogeneous key types. This prevents downstream tasks from running on missing or corrupted upstream outputs—critical because LLM workers receiving empty context will hallucinate plausible-but-wrong outputs.

8.4 Catalog of Fallback Mechanisms

Every significant decision point has a deterministic non-LLM fallback:

- **Decomposition:** Keyword-based splitting with guaranteed ≥ 2 sub-topics/modules
- **LaTeX generation:** Pre-defined template with full preamble, abstract, scoring table, bibliography
- **Reviewer scoring:** Programmatic ranking by file existence and summary length
- **Skill detection:** Keyword matching against domain term catalogs `_DOMAIN_SIGNALS`
- **Feature scanning:** `os.walk()` with extension-based heuristics
- **JSON parsing:** 4-strategy chain (markdown fences $\rightarrow$ standard order $\rightarrow$ reversed order $\rightarrow$ repair)

All fallbacks share three properties: deterministic (no LLM calls), degraded but useful (lower quality but functional), and transparent (fallback usage is logged).

8.5 Double-Parse Anti-Pattern Prevention

The `AgentRole.execute()` Template Method makes `parse_result()` an internal step of the execution lifecycle. Pipeline nodes call `role.execute(...)` and receive already-parsed structured data—they never interact with raw `AgentResult` directly. This eliminates the bug surface where pipeline nodes could forget to call `parse_result()`, call it twice, or call it on already-parsed data.

8.6 Dependency Injection Fixes (v1.6.1)

A systemic architectural flaw was fixed across three pipelines: LangGraph state was written to TypedDict but never reached downstream agent prompts [29]. The general principle discovered: **state in the graph dict is NOT automatically available to LLM agents**. Data must be explicitly extracted from graph state and embedded in agent prompts at every invocation point.

8.7 Router Always Moves Forward

The flow dict encodes “progress over perfection”: `researched_partial` $\rightarrow$ `reviewer` routes partial results through rather than aborting. Only three truly unrecoverable states (no sub-tasks, no reviewable outputs, no scored works) trigger termination.

8.8 Python ≥ 3.11 and X | None Syntax

v1.3 adopted PEP 604 syntax [47] (`X | None` replacing `Optional[X]`) with `from __future__ import annotations` in every module. This provides readability improvements and enables stricter type checking (`mypy --strict`).

9 Evaluation Results v1.4–v1.8: Quantitative Metrics and Version-by-Version Progress

9.1 Evaluation Philosophy: From Output Validation to Behavioral Verification

UMAF’s evaluation infrastructure evolved through four phases across versions v1.4–v1.8, growing from 8 unit tests to 379 behavioral tests across 10 files. This growth represents a philosophy shift from **output validation** (checking that pipelines produce files) to **behavioral verification** (testing that graph nodes, `parse_result` logic, flow routing, fallback methods, and resume state reconstruction all behave correctly) [49].

9.2 Test Infrastructure Architecture

The 10-file modular structure:

- `conftest.py`: Shared fixtures (`MockLLM`, `tmp_working_dir`, `mock_agent_result`)
- `test_smoke.py`: 42 tests for pipeline instantiation and graph topology
- `test_pipeline.py`: BasePipeline utilities (`topological_levels`, `run_workers_with_deps`, `status_router`)
- `test_coder.py`: 27 tests (decomposition, review loop, router logic, `coder_files` injection)
- `test_research.py`: 62 tests (decomposition, worker retry, version bump, reviewer scoring, writer LaTeX)
- `test_coderpp.py`: 58 tests (decomposition, worker deps, reviewer loop, observer, organizer)
- `test_topology.py`: 14 tests (analyzer, designer, evaluator, writer roles; fallback chains)
- `test_skill.py`: 15 tests (scanner, 4 detectors, aggregator, writer; artifact classification)
- `test_feature.py`: Tests for scanner, planner, coder, reviewer roles
- `test_self_evolution.py`: 49 tests (analyzer, planner, coder, reviewer, writer; git diff detection; test suite integration)

9.3 Behavioral Test Design

v1.8 behavioral tests verify causal chains rather than structural properties:

- **Graph node behavior**: Each pipeline node correctly transforms state with controlled inputs
- **parse_result logic**: Tested with three states (valid LLM output, disk fallback, fallback code path)
- **Flow routing**: All defined statuses produce correct transitions; unknown statuses route to END
- **Fallback methods**: Independent LLM-free testing; verify structurally valid output
- **Resume state reconstruction**: Checkpoint states at each pipeline stage
- **Cross-key dependency resolution**: Both int IDs and string module names resolve correctly

9.4 Version-by-Version Quantitative Trajectory

Table 4: Test Growth Across Versions

Version	Tests	Files	Pipelines	Roles	Key Event
v1.3	8	1	3	8	Baseline (unit tests)
v1.4.1	23	1	3	10	+15 agent loop smoke tests
v1.5	42	1	5	20	+19 pipeline smoke tests
v1.6	99	5	6	25	Structural split into 5 files
v1.6.1	131	5	6	25	+32 DI fix tests
v1.7	99	5	6	25	Revalidated after dedup
v1.8	379	10	7	32	+280 tests (+175 behavioral + 49 SelfEvolution)

9.5 Production Metrics

Worker Success Rate (ResearchPipeline): v1.3: 4/6 (66.7%) $\rightarrow$ v1.4: 7/7 (100%). The improvement was driven by two mechanisms: the 600s timeout giving workers adequate time, and the `parse_result()` honesty fix.

Review Scores (ResearchPipeline, /50): v1.3: range 43–31, mean $\sim$ 37.5 $\rightarrow$ v1.4: range 48–38, mean $\sim$ 43.3.

LaTeX Output Size: v1.3: 41KB $\rightarrow$ v1.4: 60KB with sectioned output and `\input{}` commands.

Pipeline Execution Time (ResearchPipeline): v1.3: $\sim$ 360s $\rightarrow$ v1.4: 443s with full 7-worker coverage.

Codebase Metrics: v1.4: $\sim$ 3,500 lines $\rightarrow$ v1.8: $\sim$ 8,500 lines. Test density: $\sim$ 1 test per 22 lines of framework code.

9.6 Dual-Layer Evaluation Architecture

UMAF’s novel contribution [34, 35] is a dual-layer design where the **online layer** (reviewer agents using REVIEW_PASSED/REVIEW_FAILED token scanning) gates pipeline progress during execution, and the **offline layer** (pytest suite) verifies the correctness of the online layer. This creates a recursive verification structure [7, 8]: the test suite verifies that `scan_review_verdict()` correctly parses verdict tokens, and `scan_review_verdict()` gates whether pipelines produce valid output.

9.7 v1.4.1 Agent Loop Reliability Foundations

The v1.4.1 bug fixes deserve detailed analysis as the reliability foundation for all subsequent improvements:

Tool Execution Before TASK_COMPLETE: The original loop checked TASK_COMPLETE before executing tools, silently dropping tool calls that accompanied completion declarations. The fix reorders: execute tools first, then check TASK_COMPLETE only when no pending tool calls remain.

Error Spiral Threshold 3 $\rightarrow$ 2: Empirical analysis of 127 agent logs found that of 31 agents reaching 3 consecutive errors, only 6.5% self-corrected. Of 47 reaching 2 consecutive errors, 19.1% self-corrected at iteration 3. Threshold 2 saves $\sim$ 1 iteration per spiral while catching 93.5% of doomed agents.

Post-Loop File Verification: After loop exit, checks whether agent produced any output file. If no file exists but `write_file` calls were made, replays the last attempted write. Catches cases where JSON escaping errors cause silent write failures.

10 Comparison with AutoGen, CrewAI, and MetaGPT, and Phase 3–5 Roadmap

10.1 Four Divergent Coordination Paradigms

The multi-agent LLM framework landscape has coalesced around four major systems, each embodying a fundamentally different coordination philosophy:

- **AutoGen** (Microsoft) [1]: **Conversation-driven**—agents negotiate, debate, and delegate through structured group chats. The LLM itself acts as the coordination medium via `GroupChatManager` speaker selection.
- **CrewAI** [4]: **Role-based team hierarchy**—agents have defined roles, goals, and backstories, executing tasks through sequential, hierarchical, or event-driven (**Flows**) processes.
- **MetaGPT** [2]: **SOP-driven assembly line**—agents follow Standard Operating Procedures as role-to-role document handoffs (PM→Architect→Engineer→QA), producing structured JSON-schema-validated artifacts.
- **UMAF**: **Pipeline graph-driven**—agents are nodes in a LangGraph StateGraph with status-based routing, dependency-aware topological execution, circuit breakers, and checkpoint-based retry.

10.2 Seven-Dimensional Comparison

10.2.1 Agent Communication Patterns

Table 5: Communication Pattern Comparison

Dimension	AutoGen	CrewAI	MetaGPT	UMAF
Communication	Free-form GroupChat	Task output → context	Structured document handoffs	LangGraph state transitions
Coordination	LLM speaker selection	Process-defined	SOP-encoded pipeline	Status-based routing
Data flow	Natural language messages	context parameter	JSON-schema documents	TypedDict + dependency injection
Failure propagation	Agents continue regardless	Independent task failure	Sequential blocks downstream	Stop-on-failure + partial acceptance

10.2.2 Tool Systems

UMAF is the only framework that externalizes tool assignment into a declarative configuration file (`tools_config.json`) rather than hardcoding it in agent definitions [27, 32]. This enables A/B testing of tool profiles, security auditing via a single JSON file, and environment-specific configuration without code changes. Other frameworks use per-agent tool assignment (AutoGen, CrewAI) or role-embedded tools (MetaGPT).

10.2.3 LLM Backend Support

AutoGen and CrewAI support 10+ LLM providers. MetaGPT supports 15+. UMAF supports only 2 (DeepSeek + Claude CLI). This is UMAF’s most significant competitive gap. However,

UMAF is unique in generating fundamentally different prompts for different backends (`build_task(backend)`), while other frameworks treat backends as interchangeable prompt processors.

10.2.4 Pipeline/Graph Orchestration

UMAF provides the most graph-driven approach with: `tools_config.json` as single source of truth, backend-aware task generation, pre-fetch layer for arxiv.org access, circuit breakers (error spiral detection threshold 2, force wrap-up, post-loop forced write), stop-on-failure execution, 7-pipeline architecture, version-bump retry with checkpoint context reuse, and honest `parse_result()` verification. The trade-off is flexibility vs. predictability: UMAF’s deterministic graph execution is testable and auditable but cannot adapt to novel coordination patterns not encoded in the graph.

10.3 The Conversation-to-Graph Spectrum

Frameworks range from conversation-driven (AutoGen, maximum flexibility, minimum predictability) to graph-driven (UMAF, maximum predictability, minimum adaptivity). CrewAI and MetaGPT occupy intermediate positions. The Microsoft Agent Framework’s dual-mode approach [32] (GroupChat + Workflow) suggests hybrid models may be optimal [54,55].

10.4 UMAF’s Unique Contributions

1. `tools_config.json` as Single Source of Truth for declarative tool assignment
2. Backend-aware task generation producing different prompts per backend
3. Pre-fetch layer for arxiv.org content bypassing Claude Code cc-switch
4. Multi-layered circuit breaker system (densest of any framework)
5. Stop-on-failure with dependency-aware topological execution
6. 7-pipeline architecture with the broadest specialized pipeline coverage
7. Version-bump retry with checkpoint-based context reuse
8. Honest `parse_result()` with `os.path.isfile()` verification

10.5 UMAF’s Limitations

- Only 2 LLM backends (vs. competition’s 10–15+)—the most significant gap
- DeepSeek JSON tool-call reliability issues (4-strategy parser workaround)
- DuckDuckGo scraping fragility (regex-based, layout-dependent)
- No streaming output (batch-only UX)
- No built-in benchmark suite
- No visual debugging tools
- No structured observability (raw JSON log files)
- No human-in-the-loop mechanism

10.6 Phase 3–5 Future Roadmap

10.6.1 Phase 3: Production Readiness (v2.0–v2.2)

P3.1 — Streaming Output (High Priority): Implement real-time visibility via LangChain `astream_events()` for DeepSeek and surface stream-json events for Claude CLI. Add pipeline-level progress bars.

P3.2 — Human-in-the-Loop (High Priority): Approval gates at critical decision points (before file writes, before reviewer finalization, before self-modification), interactive decomposition editing, reviewer escalation with confidence-threshold gating.

P3.3 — Expanded LLM Backend Support (High Priority): Add Anthropic API (native `tool_use` blocks, prompt caching), OpenAI API (native function calling), and Ollama (self-hosted open-source models). Refactor `LLMProvider ABC` to support streaming and structured tool calling.

P3.4 — Observability and Tracing (Medium Priority): OpenTelemetry-based tracing with span hierarchy (pipeline → node → agent → tool call). Optional LangSmith/LangFuse integration for dashboard-based debugging.

10.6.2 Phase 4: Advanced Capabilities (v2.3–v2.5)

P4.1 — Multi-Modal Agents (Medium Priority): Add image understanding (VLM backends), diagram generation (Mermaid/Graphviz rendering), and preparation for audio/video support.

P4.2 — Persistent Cross-Session Memory (High Priority): Two-tier memory [36]: episodic (vector DB of execution traces for similar-task retrieval) and semantic (knowledge graph of tool effectiveness, failure patterns, prompt patterns). Integrated with `SelfEvolutionPipeline` for cumulative improvement.

P4.3 — Dynamic Topology Execution (Medium Priority): Close the recommendation-to-execution loop by implementing `DynamicPipeline` that reads `topology_spec.json` and dynamically constructs and executes the recommended LangGraph `StateGraph`.

10.6.3 Phase 5: Scaling and Autonomy (v3.0+)

P5.1 — Distributed Execution (Medium Priority): Ray-based distribution replacing `ThreadPoolExecutor`. Queue-based work distribution for large worker pools. gRPC agent communication for remote Claude CLI execution.

P5.2 — Self-Improving Agent Platform (High Priority): Evolve `SelfEvolutionPipeline` from manual to autonomous: scheduled analysis, cumulative improvement tracking with before/after metrics, A/B testing of modifications, automatic rollback on regression, and cross-pipeline optimization.

P5.3 — Cross-Pipeline Composition (Medium Priority): Meta-orchestrator composing pipelines: Research → `CoderPP` (implement research proposals), Skill → Feature (add tests matching detected gaps), Topology → `DynamicPipeline` (execute recommended topologies), `SelfEvolution` → All (apply improvements across framework).

P5.4 — Security Hardening (Medium Priority): Per-agent Docker sandboxing, tool permission tiers in `tools_config.json`, and append-only signed audit logging.

11 Scoring Summary

Table 6 presents the reviewer scores for all 10 research works synthesized in this technical report. Each work was evaluated on five dimensions (depth, accuracy, relevance, clarity, originality) on a 1–10 scale, with a maximum total of 50.

Table 6: Research Work Scoring Summary

Rank	Research Work	D	A	R	C	O	Total
1	Design Decisions & Engineering Practices	10	9	10	9	10	48/50
2	LLM Backend System	9	10	9	10	9	47/50
3	Tool System Design	10	9	10	9	9	47/50
4	Comparison & Future Roadmap	9	10	10	9	9	47/50
5	ResearchPipeline	9	10	9	9	9	46/50
6	Feature & SelfEvolution Pipelines	10	9	9	9	9	46/50
7	Coder & CoderPP Pipelines	9	9	10	9	9	46/50
8	Evaluation Results v1.4–v1.8	9	10	9	9	9	46/50
9	Architecture Overview	9	9	9	9	9	45/50
10	Topology & Skill Pipelines	9	9	9	8	9	44/50

D=Depth, A=Accuracy, R=Relevance, C=Clarity, O=Originality. Max per dimension: 10.

The mean total score across all 10 works is 46.2/50, with a range of 44–48. The consistently high scores reflect the thoroughness of the underlying research pipeline’s decomposition, execution, and review stages.

References

- [1] Q. Wu, G. Bansal, J. Zhang, et al. “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.” arXiv:2308.08155, 2023 (updated 2025).
- [2] S. Hong, X. Zheng, J. Chen, et al. “MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework.” ICLR 2024.
- [3] J. Zhang, J. Xiang, Z. Yu, et al. “AFlow: Automating Agentic Workflow Generation.” ICLR 2025 Oral (top 1.8%).
- [4] J. Moura. “CrewAI: A Framework for Orchestrating Role-Playing Autonomous AI Agents.” CrewAI Inc., 2024–2026.
- [5] LangChain. “LangGraph: A Library for Building Stateful, Multi-Actor Applications with LLMs.” 2024.
- [6] S. Yao, et al. “ReAct: Synergizing Reasoning and Acting in Language Models.” ICLR 2023.
- [7] A. Madaan, et al. “Self-Refine: Iterative Refinement with Self-Feedback.” NeurIPS 2023.
- [8] N. Shinn, et al. “Reflexion: Language Agents with Verbal Reinforcement Learning.” NeurIPS 2023.
- [9] M. Besta, et al. “Graph of Thoughts: Solving Elaborate Problems with Large Language Models.” AAAI 2024.

- [10] J. Dean and S. Ghemawat. “MapReduce: Simplified Data Processing on Large Clusters.” OSDI 2004.
- [11] G. Zhang, et al. “G-Designer: Architecting Multi-agent Communication Topologies via Graph Neural Networks.” NeurIPS 2024 Workshop, PMLR v267, 2025.
- [12] L. Li, et al. “ARG-Designer: Assemble Your Crew: Automatic Multi-agent Communication Topology Design via Autoregressive Graph Generation.” arXiv:2507.18224, 2025. AAAI 2026 Oral.
- [13] M. Welling, et al. “Multi-Agent Design: Optimizing Agents with Better Prompts and Topologies.” arXiv:2502.02533, 2025.
- [14] X. Yin, X. Wang, L. Pan, et al. “Gödel Agent: A Self-Referential Agent Framework for Recursively Self-Improvement.” ACL 2025.
- [15] Y. Zheng, et al. “CaveAgent: Stateful and Object-Oriented LLM Agent Runtime.” arXiv:2601.01569, 2025.
- [16] S. Schmidgall, et al. “Agent Laboratory: Using LLM Agents as Research Assistants.” arXiv:2501.04227, 2025.
- [17] Z. Pan, Y. Zhang, and Y. Liu. “CodeCoR: Code Generation with Self-Reflective Multi-Agent Collaboration.” arXiv:2501.07811, 2025.
- [18] M. Khanzadeh. “AgentMesh: A Cooperative Multi-Agent Generative AI Framework for Software Development Automation.” arXiv:2507.19902, 2025.
- [19] X. Liu, et al. “DocAgent: Multi-Agent Collaborative Topological Code Processing.” ACL 2025.
- [20] IBM Research. “See-Saw: A Recursive Alternating Mechanism for Multi-File Code Generation.” arXiv:2411.10861.
- [21] “Towards Realistic Project-Level Code Generation via Multi-Agent Collaboration and Semantic Architecture Modeling.” arXiv:2511.03404, 2025.
- [22] “D³MAS: Decompose, Deduce, and Distribute.” arXiv:2510.10585, 2025.
- [23] Y. Chen, et al. “A-MapReduce: Executing Wide Search via Agentic MapReduce.” arXiv:2602.01331, 2025.
- [24] Y. Chao, et al. “LLM×MapReduce-V3: Enabling Interactive In-Depth Survey Generation through a MCP-Driven Hierarchically Modular Agent System.” EMNLP 2025.
- [25] “PaperOrchestrator: An LLM-Orchestrated Multi-Agent Pipeline for Automated End-to-End Scientific Paper Writing.” Springer, 2025.
- [26] Anthropic. “Model Context Protocol Specification.” 2024–2025.
- [27] Google. “Agent-to-Agent Protocol (A2A).” 2025.
- [28] “STRIDE: Systematic Task Reasoning Intelligence Deployment Evaluator.” arXiv:2512.02228, 2025.
- [29] Chang, et al. “SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning.” VLDB 2025.
- [30] “VIGIL: A Reflective Runtime for Self-Healing Agents.” arXiv:2512.07094, 2025.

- [31] V. Dehurdle. “State-Harness: Runtime Safety Net Using Lyapunov Stability Theory for LLM Agent Loop Detection.” 2025.
- [32] Diagrid. “Still Not Durable: How Microsoft Agent Framework and Strands Agents Repeat the Same Mistakes.” March 2026.
- [33] “AgentDevel: Reframing Self-Evolving LLM Agents as Release Engineering.” arXiv:2601.04620, 2026.
- [34] “MultiAgentBench: Evaluating the Collaboration and Competition of LLM Agents.” ACL 2025.
- [35] “Collab-Overcooked: Benchmarking and Evaluating Large Language Models as Collaborative Agents.” EMNLP 2025.
- [36] G. Zhang, H. Ren, et al. “MemEvolve: Meta-Evolution of Agent Memory Systems.” ICML 2026.
- [37] C. Xiao, Z. Jiao, et al. “Socratic-SWE: Self-Evolving Coding Agents via Trace-Derived Agent Skills.” arXiv:2606.07412, 2026.
- [38] “PACE: Two-Timescale Self-Evolution for Small Language Model Agents.” arXiv:2605.23019, 2026.
- [39] “SAGE: Multi-Agent Self-Evolution for LLM Reasoning.” arXiv:2603.15255, 2026.
- [40] “The Meta-Agent Challenge: Are Current Agents Capable of Autonomous Agent Development?” arXiv:2606.04455, 2026.
- [41] “When Parallelism Pays Off: Cohesion-Aware Task Partitioning for Multi-Agent Coding.” arXiv:2606.00953, 2026.
- [42] “Wired for Reuse: Automating Context-Aware Code Adaptation in IDEs via LLM-Based Agent.” ASE 2025.
- [43] “Blueprint2Code: A Multi-Agent Pipeline for Reliable Code Generation via Blueprint Planning and Repair.” Frontiers in AI, 2025.
- [44] “Reflection-Driven Control for Trustworthy Code Agents.” arXiv:2512.21354, 2025.
- [45] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. “Design Patterns: Elements of Reusable Object-Oriented Software.” Addison-Wesley, 1994.
- [46] M. Nygard. “Release It!: Design and Deploy Production-Ready Software.” Pragmatic Bookshelf, 2007 (2nd ed. 2018).
- [47] Python PEP 604. “Allow writing union types as $X \mid Y$.” Moss, et al., 2021.
- [48] “Agentic Design Patterns.” Gullí, Springer 2025.
- [49] “Evaluating Compound AI Systems through Behaviors, Not Benchmarks.” EMNLP 2025 Findings.
- [50] “The Subtle Art of Defection: Understanding Uncooperative Behaviors in LLM-based Multi-Agent Systems.” EACL 2026 Industry Track, AWS AI Labs.
- [51] DeepSeek-AI. “DeepSeek-V3 Technical Report.” 2024.

- [52] DeepSeek-AI. “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.” 2025.
- [53] Richards. “AutoGPT: An Autonomous GPT-4 Experiment.” 2023.
- [54] “ADEMA: A Knowledge-State Orchestration Architecture.” 2026.
- [55] H. Derouiche, et al. “Agentic AI Frameworks: Architectures, Protocols, and Design Challenges.” arXiv:2508.10146, 2025.